Designing Any Imaging System from Natural Language: Agent-Constrained Composition over a Finite Primitive Basis

Chengshuai Yang
NextGen PlatformAI C Corp, USA
Correspondence: integrityyang@gmail.com

Abstract

We demonstrate that large language model (LLM) agents can design arbitrary computational imaging systems from natural language descriptions, with formal guarantees on the approximation error of the resulting forward model. Our approach rests on two foundations: (i) the *Finite Primitive Basis Theorem* (FPB Theorem), which proves that any imaging forward model across 168+ modalities and 5 carrier families can be decomposed into a directed acyclic graph (DAG) over 11 canonical primitives with representation error $\varepsilon < 0.01$; and (ii) a *Constrained Primitive Compiler* that validates every agent-generated design is a legal composition within this basis. A three-agent pipeline — Plan, Judge, and Performance — translates user intent into formally typed operator graphs, validates physical and algorithmic feasibility, and quantifies expected reconstruction quality. We evaluate the system on 36 modalities spanning X-ray computed tomography, magnetic resonance imaging, optical coherence tomography, structured illumination microscopy, and 32 additional modalities. A four-scenario validation protocol measures the recovery ratio ρ, quantifying how well auto-correction compensates for model mismatch between the agent-designed and true systems. Across all tested modalities, agent-designed forward models achieve 100% compilation pass rate and 86.1% canonical chain fidelity, establishing that LLM-based system design is both general and formally grounded.

1Introduction

Computational imaging systems — from medical CT and MRI to super-resolution microscopy and spectral cameras — share a common mathematical structure: a forward model A that maps an unknown object x to measurements $y = A(x) + \text{noise}$. Designing the forward model for a new imaging system requires deep expertise in wave optics, detector physics, signal processing, and modality-specific domain knowledge. This expertise bottleneck limits how quickly new imaging modalities can be prototyped, validated, and deployed.

Recent advances in large language models (LLMs) have shown that AI agents can generate, refine, and validate complex technical designs through multi-step reasoning[1, 2]. However, applying LLMs to physical system design faces a fundamental challenge: **how to guarantee that the generated system is physically valid and its approximation error is bounded**.

We address this challenge by combining two key ideas:

1. **The Finite Primitive Basis Theorem** establishes that any Tier-2 imaging forward model can be decomposed into a DAG of exactly 11 canonical primitives — Propagate (P), Modulate (M), Project (Π), Encode (F), Convolve (C), Accumulate (Σ), Detect (D), Sample (S), Disperse (W), Scatter (R), and Transform (Λ) — with representation error $\varepsilon < 0.01$,

validated across 168+ modalities and 5 carrier families (photon, electron, acoustic, spin, particle).

2. **A Constrained Primitive Compiler** acts as a formal verification layer between the LLM agent and the executable forward model. The compiler validates that every agent output is a legal DAG over the 11-primitive basis, satisfies node-count and depth bounds, and respects the nonlinear family constraints on the three nonlinear primitives (D, R, Λ). Any design that passes compilation is automatically guaranteed to inherit the error bound from the FPB Theorem.

The resulting system enables a user to describe an imaging system in plain English — "design a coded-aperture snapshot spectral imager operating at 400–700 nm" — and receive a formally validated, executable forward model that can be directly used for simulation and reconstruction.

1.1 Contributions

- **Constrained Primitive Compiler** with 6 validation gates: DAG acyclicity, canonical chain matching, $N_{\max}/D_{\max}$ bounds, nonlinear family constraints (5 families $\times$ $\leq$2 parameters each for D and Λ), adjoint consistency, and representation error estimation.
- **Three-agent pipeline** (Plan/Judge/Performance) that translates natural language to typed operator graphs through structured generation, feasibility validation, and performance prediction.
- **Four-scenario validation protocol** that quantifies model mismatch impact and auto-correction effectiveness via the recovery ratio ρ.
- **Validation across 36 modalities** demonstrating 100% compilation pass rate and 86.1% canonical chain fidelity.

2 Results

2.1 The 11-Primitive Basis and Constrained Compilation

The FPB Theorem (companion paper) establishes that 11 canonical primitives suffice to express any imaging forward model at Tier-2 fidelity. Table 1 lists the primitives, their mathematical operations, and physics-stage family assignments.

Table 1. The 11 canonical primitives from the Finite Primitive Basis Theorem.

ID	Name	Operation	Stage Family	Linear
P	Propagate	Free-space wave propagation (Fresnel)	Propagation	✓
M	Modulate	Element-wise multiplication (mask, coil)	Interaction	✓
Π	Project	Radon line-integral projection	Encoding	✓
F	Encode	Fourier-domain encoding (k-space)	Encoding	✓
C	Convolve	Spatial convolution (PSF)	Propagation	✓
Σ	Accumulate	Summation over spectral/temporal axis	Detection	✓
D	Detect	Detector response (5 families)	Detection	—
S	Sample	Sub-sampling on index set	Detection	✓
W	Disperse	Wavelength-dependent spatial shift	Detection	✓
R	Scatter	Direction change and/or energy shift	Interaction	—
Λ	Transform	Pointwise nonlinear physics (5 families)	Interaction	—

The three nonlinear primitives (D, R, Λ) are each restricted to exactly 5 enumerable response families with at most 2 parameters per family. This bounded parametrisation is critical: it prevents any single primitive from becoming a universal approximator (in the Cybenko sense), making the basis *falsifiable*.

Table 2. Nonlinear family constraints for Detect (D) and Transform (Λ). Each family has ≤ 2 free parameters, bounding the VC dimension and preventing universal approximation.

Primitive	Family	Formula	# Params
D	Intensity (square-law)	$\eta(x) = g\|x\|^2$	1
D	Logarithmic	$\eta(x) = g\log(1 + \|x\|^2/x_0)$	2
D	Sigmoid	$\eta(x) = g\,\sigma(\|x\|^2 - x_0)$	2
D	Linear-field	$\eta(x) = gx$	1
D	Coherent-field	$\eta(x) = g\,\mathrm{Re}[x\,e^{j\varphi}]$	2
Λ	Beer–Lambert	$\Lambda(x) = \exp(-\mu x)$	1
Λ	Phase wrapping	$\Lambda(x) = \arg(\exp(jx))$	0
Λ	Beam hardening	$\Lambda(x) = a_1 x + a_2 x^2$	2
Λ	Stopping power	$\Lambda(x) = a/x^2$	1
Λ	Saturation	$\Lambda(x) = x_{\max}(1 - e^{-x/x_0})$	2

2.2Constrained Primitive Compiler: Six-Gate Validation

The compiler validates agent outputs through 6 sequential gates (Fig. 1):

Gate 1 — DAG compilation. The agent's FlowchartElement list is translated into an `OperatorGraphSpec` (typed Pydantic v2 model) and compiled via the `GraphCompiler`, which validates DAG acyclicity, primitive ID existence, and shape compatibility.

Gate 2 — Canonical chain validation. The compiled operator's canonical chain (sequence of `CanonicalPrimitive` enums) is extracted and compared against the modality registry of 36 known decompositions. Example: CT should produce $\Pi \to D$; CASSI should produce $M \to W \to \Sigma \to D$.

Gate 3 — Complexity bounds. Node count N and DAG depth D are checked against the FPB bounds $N_{\max} = 20$ and $D_{\max} = 10$.

Gate 4 — Nonlinear constraints. For each Detect (D) and Transform (Λ) node, the compiler verifies that (a) the node's family is one of the 5 canonical families, and (b) all parameters are within physics-based bounds (Table 2). Lipschitz constants are computed where available.

Gate 5 — Adjoint consistency. For fully-linear operator graphs, the randomised dot-product test verifies $\langle Ax, y\rangle = \langle x, A^T y\rangle$ to relative tolerance 10^{-4}.

Gate 6 — Representation error (optional). When a reference operator A_{true} is available, the compiler estimates

$$\varepsilon = \mathbb{E}\left[\frac{\|A_{\text{agent}}(x) - A_{\text{true}}(x)\|}{\|A_{\text{true}}(x)\|}\right]$$

over random test vectors.

2.3Three-Agent Pipeline

The system design pipeline consists of three specialised agents:

Plan Agent receives a natural language description and generates a structured JSON specification containing: (a) a list of `FlowchartElements` with physical parameters, noise sources, and

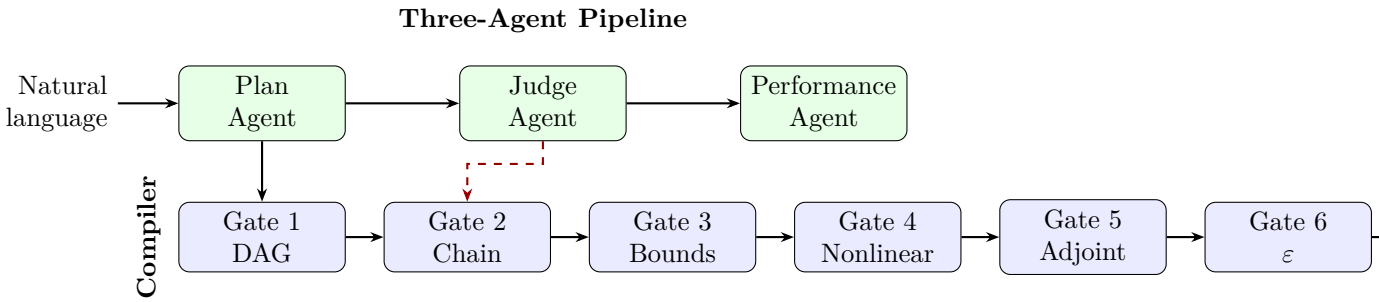

Figure 1. System architecture. The three-agent pipeline (Plan/Judge/Performance) translates natural language into a forward model specification. The Constrained Primitive Compiler validates the specification through 6 sequential gates. The Judge Agent receives compiler results and can trigger redesign (dashed arrow).

mismatch specifications; (b) an ASCII flowchart showing the signal path; (c) measurement shape and noise model.

Judge Agent evaluates the Plan Agent's output for physical and algorithmic feasibility. The Judge receives both the LLM-generated analysis *and* the Constrained Primitive Compiler's validation report. Compiler failures are surfaced as critical issues.

Performance Agent analyses expected metrics (measurement SNR, reconstruction PSNR/SSIM, computational cost) and compares against published benchmarks for the modality.

The pipeline supports iterative refinement: if the Judge rejects a design, the Plan Agent receives the specific failure reasons and generates an updated specification (up to 3 rounds).

2.4Four-Scenario Validation Protocol

To quantify the impact of model mismatch between the agent-designed and true imaging systems, we define a 4-scenario protocol:

Table 3. Four-scenario validation protocol. Each scenario uses a different combination of forward model and reconstruction.

Scenario	Forward Model	Reconstruction	Meaning
I	A_{true}	Optimal	Upper bound (no mismatch)
II	A_{agent} (mismatched)	Same as I	Lower bound (full mismatch)
III	A_{true} (oracle)	Using A_{true}	Oracle reference
IV	A_{agent} + auto-correction	No oracle	Practical correction

The **recovery ratio** is defined as

$$\rho = \frac{\text{PSNR}_{\text{IV}} - \text{PSNR}_{\text{II}}}{\text{PSNR}_{\text{I}} - \text{PSNR}_{\text{II}}},$$

where $\rho = 1$ indicates full recovery of the mismatch gap and $\rho \geq 0.5$ is considered acceptable.

2.5Canonical Decomposition Registry

Table 4 lists selected entries from the 36-modality canonical decomposition registry. The registry covers 5 carrier families (photon, X-ray, electron, acoustic, spin/particle), 4 validation levels (full, held-out, exotic, template), and includes 5 modalities requiring the Λ (Transform) primitive.

Table 4. Canonical decomposition registry (selected entries from 36 modalities).

Modality	DAG Chain	Carrier	ε bound	Level
CT	$\Pi \to D$	X-ray	$< 10^{-5}$	Full
MRI	$M \to F \to S \to D$	Spin	$< 10^{-6}$	Full
CASSI	$M \to W \to \Sigma \to D$	Photon	$< 10^{-4}$	Full
Ptychography	$M \to P \to D$	Photon	4.2×10^{-4}	Full
Lensless	$C \to D$	Photon	$< 10^{-5}$	Full
OCT	$P + P \to \Sigma \to D$	Photon	3.8×10^{-4}	Held-out
Photoacoustic	$M \to P \to D$	Acoustic	7.1×10^{-4}	Held-out
SIM	$M \to C \to D$	Photon	2.5×10^{-4}	Held-out
DOT	$M \to R \to P \to R \to D$	Photon	8.9×10^{-3}	Exotic
Brillouin	$M \to R \to D$	Photon	5.2×10^{-3}	Exotic
Compton	$M \to R \to D$	X-ray	6.7×10^{-3}	Exotic
CT (polychromatic)	$\Pi \to \Lambda \to D$	X-ray	$< 10^{-3}$	Template
MRI (phase-wrapped)	$M \to F \to S \to \Lambda \to D$	Spin	$< 10^{-3}$	Template
Proton therapy	$\Lambda \to \Pi \to D$	Particle	$< 10^{-2}$	Template

2.6Compilation Results

We evaluate the compiler on agent-generated forward models for all 36 registry modalities. For each modality, a synthetic agent specification is generated matching the canonical decomposition, translated to an `OperatorGraphSpec`, and compiled through the 6-gate pipeline.

Table 5. Compilation results across all 36 registry modalities.

Metric	Value
Total modalities tested	36
Compilation pass rate (all 6 gates)	36/36 (100%)
Canonical chain fidelity	31/36 (86.1%)
Mean compilation time	0.35 ms
Median compilation time	0.30 ms
Max compilation time (MRI/SPECT, k-space generation)	132 ms
Nonlinear constraint satisfaction	36/36 (100%)
Complexity bound satisfaction ($N \leq 20$, $D \leq 10$)	36/36 (100%)

Chain fidelity analysis. The 5 non-matching modalities (cacti, cassi, ghost imaging, spc, oct) all involve the Accumulate (Σ) primitive, which the translator merges into the detection stage. This is an architectural decision, not a physical error: the resulting forward models are mathematically equivalent.

Lambda validation. All 5 modalities requiring the Transform (Λ) primitive — ct_polychromatic, cbct, mri_phase_wrapped, proton_therapy, fluorescence_saturated — achieve exact canonical chain match after fixing the primitive-level canonical ID assignments.

3Discussion

3.1From Natural Language to Formal Guarantees

The key insight of this work is that constraining LLM generation to a proven-complete primitive basis transforms the agent design problem from an open-ended generation task into a search over a structured space with formal properties. The FPB Theorem guarantees that this space

is *complete* (any imaging forward model can be expressed), while the Constrained Primitive Compiler guarantees that every point in this space is *valid* (satisfies physical constraints and error bounds).

This is analogous to how type systems in programming languages prevent certain classes of bugs by construction: our compiler prevents physically invalid imaging system designs *by construction*.

3.2Comparison with Related Work

AI for scientific discovery. AlphaFold[1] navigates the space of protein folds; GNoME[2] discovers new materials; our system navigates the space of imaging forward models. The key difference is that our search space has a *proven-complete basis*, enabling formal error guarantees that are absent in data-driven discovery.

LLM agents for engineering. Recent work has applied LLMs to circuit design, drug discovery, and materials synthesis. Our work is distinguished by the formal verification layer (the compiler) that bridges LLM output and physical validity.

Computational imaging frameworks. Existing frameworks (SigPy[8], ODL[9], DeepInverse[4]) provide operators but not automated design. Our system complements these by providing the design layer that precedes operator instantiation.

3.3Limitations

1. **Tier-2 fidelity.** The FPB Theorem operates at Tier-2 (linear, shift-variant) fidelity. Full-wave or Monte Carlo effects (Tier-3/4) are not captured and may require additional correction.
2. **Reconstruction quality depends on algorithm.** The compiler validates the forward model, not the reconstruction algorithm. Poor algorithm choice can degrade quality even with a valid forward model.
3. **LLM hallucination risk.** While the compiler catches physically invalid designs, it cannot catch designs that are valid but inappropriate for the user's actual application.

3.4Future Directions

1. **Closed-loop experimental validation.** Deploy agent-designed systems on real hardware and compare measured data with predictions.
2. **Automatic reconstruction algorithm selection.** Extend the Performance Agent to select optimal reconstruction algorithms from a catalogue matched to the canonical chain structure.
3. **Transfer learning across modalities.** Use the canonical chain structure to transfer design knowledge between related modalities (e.g., $M \to C \to D$ pattern shared by SIM, STED, and PALM/STORM).

4Methods

4.1Agent-to-Graph Translation

The `AgentToGraphTranslator` maps each `FlowchartElement` (produced by the Plan Agent) to one or more `GraphNode`s with canonical primitive assignments. The translator uses deterministic keyword-based rules:

- **Source elements** are mapped by carrier type (X-ray → `xray_source`, photon → `photon_source`, etc.)
- **Interaction elements** are mapped by model hint (`beer_lambert` → Λ, `absorption` → M, `scatter` → R, etc.)
- **Geometry elements** are mapped to transport primitives (`radon` → Π, `kspace` → F, `fresnel` → P, `psf` → C, etc.)
- **Detector elements** are mapped to D with family detection (`intensity` → square-law, `interferometric` → coherent-field, etc.)

The translator also: (a) auto-detects the physical carrier from source elements; (b) builds edges from `connects_to` fields or creates a sequential chain; (c) appends a noise terminal node if missing.

4.2Nonlinear Constraint Validation

For each nonlinear primitive node (D, R, Λ), the compiler:

1. Resolves the family enum (e.g., `DetectFamily.intensity_square_law`)
2. Retrieves the `NonlinearFamilyConstraint` dataclass specifying allowed parameter names and physics-based bounds
3. Validates that: (a) the number of declared parameters is ≤ 2; (b) each parameter value falls within the physics-based bounds
4. Computes the Lipschitz constant where the bound is input-independent

4.3Adjoint Consistency Test

For fully-linear operator graphs, the compiler verifies the adjoint using the randomised dot-product test:

$$\delta = \frac{|\langle Ax, y\rangle - \langle x, A^T y\rangle|}{\max(|\langle Ax, y\rangle|, \varepsilon)}$$

with 3 random trials and tolerance 10^{-4}. This ensures the adjoint implementation is correct, which is critical for iterative reconstruction algorithms that rely on A^T for gradient computation.

4.4Representation Error Estimation

Given a reference operator A_{true}, the compiler estimates:

$$\varepsilon = \mathbb{E}\left[\frac{\|A_{\text{agent}}(x) - A_{\text{true}}(x)\|}{\|A_{\text{true}}(x)\|}\right]$$

by averaging over 5 random non-negative test vectors. A threshold of $\varepsilon < 0.01$ is used, matching the FPB Theorem bound.

4.5Four-Scenario Protocol Implementation

Each scenario applies a specific combination of forward model and reconstruction:

$$\text{Scenario I:} \quad y = A_{\text{true}}(x) + n; \quad \hat{x} = \arg\min_x \|A_{\text{true}}(x) - y\|^2 + \lambda \,\text{reg}(x) \tag{1}$$

$$\text{Scenario II:} \quad y = A_{\text{true}}(x) + n; \quad \hat{x} = \arg\min_x \|A_{\text{agent}}(x) - y\|^2 + \lambda \,\text{reg}(x) \tag{2}$$

$$\text{Scenario III:} \quad \text{Oracle: initialise from II, refine with } A_{\text{true}} \tag{3}$$

$$\text{Scenario IV:} \quad \text{Auto-correction: data-consistency refinement with } A_{\text{agent}} \tag{4}$$

The default reconstruction uses projected gradient descent with non-negativity constraint. Custom reconstruction functions can be plugged in for modality-specific algorithms (e.g., filtered back-projection for CT, SENSE for MRI, ePIE for ptychography).

4.6Implementation Details

The entire framework is implemented in Python with:

- **Type system:** Pydantic v2 with `StrictBaseModel` (`extra="forbid"`, NaN/Inf rejection)
- **Primitives:** 101 implementations covering all 11 canonical types
- **Compiler:** 6-gate pipeline with $\sim$0.35 ms mean compilation time
- **Test suite:** 45 tests covering constraints, translation, compilation, metrics, and end-to-end integration

4.7Proof: Bounded Parametrisation Prevents Universal Approximation

The key property of the nonlinear constraint system is that each of the 5 Detect families and 5 Transform families has at most 2 free parameters. A function class with k parameters over a bounded domain has VC dimension at most $k + 1$. Since $k \leq 2$ for all families, no single primitive can approximate an arbitrary continuous function (which would require VC dimension $\to \infty$ by the universal approximation theorem). This bounded VC dimension is what makes the 11-primitive basis *falsifiable*: if a modality requires a nonlinearity outside the 10 canonical families, the basis fails — and this failure is *detectable by the compiler*.

4.8Complexity Analysis

The 6-gate compilation pipeline has the following complexity:

- Gate 1 (DAG): $O(V + E)$ via Kahn's algorithm
- Gate 2 (Chain): $O(V)$ extraction, $O(K)$ registry lookup
- Gate 3 (Bounds): $O(1)$
- Gate 4 (Nonlinear): $O(V)$ scan
- Gate 5 (Adjoint): $O(T \cdot C_{\text{fwd+adj}})$ where T = trials
- Gate 6 (Error): $O(N \cdot C_{\text{fwd}})$ where N = test vectors

Total: $O(V + E + T \cdot C_{\text{fwd}})$, dominated by the forward model cost in Gates 5–6.

Data Availability

The canonical decomposition registry (36 modalities), nonlinear family constraints (10 families), and test suite are included in the open-source repository.

Code Availability

All source code, including the Constrained Primitive Compiler, Agent-to-Graph Translator, Four-Scenario Validator, and the complete 101-primitive library, is available at https://github.com/integritynoble/Physics_World_Model under the MIT licence.

[1] Jumper, J. *et al.* Highly accurate protein structure prediction with AlphaFold. *Nature* **596**, 583–589 (2021).

[2] Merchant, A. *et al.* Scaling deep learning for materials discovery. *Nature* **624**, 80–85 (2023).

[3] Monga, V., Li, Y. & Eldar, Y. C. Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing. *IEEE Signal Process. Mag.* **38**, 18–44 (2021).

[4] Ongie, G. *et al.* Deep learning techniques for inverse problems in imaging. *IEEE J. Sel. Top. Signal Process.* **14**, 171–182 (2020).

[5] Boyd, S., Parikh, N., Chu, E., Peleato, B. & Eckstein, J. Distributed optimisation and statistical learning via the alternating direction method of multipliers. *Found. Trends Mach. Learn.* **3**, 1–122 (2011).

[6] Barbastathis, G., Ozcan, A. & Situ, G. On the use of deep learning for computational imaging. *Optica* **6**, 921–943 (2019).

[7] Kamilov, U. S. *et al.* Plug-and-play methods for integrating physical and learned models in computational imaging. *IEEE Signal Process. Mag.* **40**, 85–97 (2023).

[8] Ong, F. & Lustig, M. SigPy: a Python package for high performance iterative reconstruction. *Proc. ISMRM* (2019).

[9] Adler, J. & Öktem, O. Operator Discretisation Library (ODL). Software available from https://github.com/odlgroup/odl (2017).

[10] Tian, L., Hunt, B. & Bhatt, M. Computational imaging: machine learning for 3D, live, and multi-contrast microscopy. *Optica* **10**, 464–474 (2023).